

Symbolic Provenance-Gated Defense against Indirect Prompt Injection

Pre-Final Project Progress Report

Murtazin Aizat

July 17, 2026

Code repository:

<https://github.com/muitiiifruckt/neural-taint-tracking-thesis>

Project site (diagrams, full results):

<https://muitiiifruckt.github.io/neural-taint-tracking-thesis/>

Summary

Following reviewer feedback, the project is narrowed and reframed as a **symbolic provenance-gated defense** (not “neural taint tracking”), focused on stages T1–T3. The research question is: *does field-sensitive provenance gating (T3) reduce targeted attack success rate (ASR) while preserving benign utility better than a strict gate (T2)?* A learned sanitizer and a formal noninterference proof are explicitly out of scope.

Since the previous stage, all four treatments T0–T3 are now fully implemented, unit-tested, and evaluated end-to-end on the AgentDojo benchmark. This report presents the complete T0–T3 comparison and answers the research question.

1 What Has Been Done So Far

Repository and reproducibility. All code is committed and pushed (the previous submission’s gap): `src/taint/` (taint core + gate policy + AgentDojo pipeline integration), `src/agent/` (pipeline wrapper + shared metric arithmetic), `tests/` (29 unit tests, no network needed), `scripts/`, `docs/` (per-stage specifications), and `results/` (summaries + per-case taint traces). Dependencies are pinned; the evaluation runner asserts the expected case counts on every full run. A themed documentation site presents the diagrams and results.

Three separated evaluations. The benchmark reporting is split into three evaluations that must never be mixed into one number:

1. *Benign user-task evaluation* — 10 user tasks, no attack.
2. *Direct injection-goal achievability* — the 14 injection goals executed directly as ordinary requests: **7 of 14** are achievable when simply asked.
3. *Indirect prompt-injection security* — 140 attacked cases. The **primary** ASR metric is computed on the direct-achievable subset (70 cases); the full-set ASR (140 cases) is secondary, since the full set mixes in 7 goals the agent cannot perform anyway (attacks on them succeed in 0/70 cases).

Corrected metric semantics. While wiring the T2 gate, a sign error was found in the metric aggregation: AgentDojo’s raw per-case `security` field is `True` when the *injection goal was achieved* (confirmed by the `BaseInjectionTask.security` docstring, by `TaskSuite.check` which runs the attack’s ground truth and requires `True`, and empirically on the logs). Earlier scripts read it inverted, so all previously reported ASR values were the complement of the true ones (e.g. the earlier “75.7% ASR” is in fact a 75.7% *security rate*, i.e. 24.3% ASR). The arithmetic is now centralized in `src/agent/metrics.py` and guarded by a regression test that reproduces exactly this inversion. All numbers below use the corrected semantics.

T1 — tag-only taint tracking. A forked tools executor (`ObservingToolsExecutor`) is integrated into a live AgentDojo pipeline; it labels the *raw structured* tool output before it is flattened to a string, preserving per-field provenance. Real traces (150 files) show untrusted tool outputs propagating into later tool-call arguments with parent lineage. Running the real benchmark also surfaced and fixed a genuine matching bug (short values extracted from a larger untrusted block were not detected).

T2 / T3 — the gate. `src/taint/policy.py` defines an explicit list of 10 privileged (state-changing) tools and a per-tool table of sensitive fields (routing / exfiltration targets and destructive-target ids are sensitive; content fields are not). `GatingToolsExecutor` blocks a privileged call — the call is *not executed*, the model receives an explanatory tool error, and the trace records the block reason and the exact tainted argument paths. T2 blocks on any tainted argument; T3 blocks only on tainted sensitive fields.

2 Results: T0–T3 Comparison

All numbers are on the same 150 cases (10 benign + 140 security), same model (gpt-4o-mini-2024-07-18), suite `workspace` (v1.2.2), attack `important_instructions`.

Table 1: Metric comparison across stages. Targeted ASR (subset) is the primary metric.

Metric	T0	T1	T2 (strict)	T3 (field)
Benign Utility	100.0%	100.0%	80.0%	80.0%
Utility Under Attack	20.0%	20.0%	34.3%	36.4%
Targeted ASR, subset (primary)	48.6%	51.4%	11.4%	8.6%
Targeted ASR, full set	24.3%	28.6%	5.7%	4.3%
Benign overblock rate	—	0%	20.0%	20.0%
Blocked calls (total)	—	0	579	496
Taint leakage rate	—	36.8%	0.0%	2.1%

T0 → T1. Benign Utility and Utility Under Attack are *identical* to T0 — the acceptance criterion for a purely observational layer. The small ASR difference is run-to-run LLM noise; the gate is disabled and cannot affect outcomes. Taint leakage of 36.8% means over a third of executed privileged calls carried untrusted content — the direct motivation for a gate.

T1 → T2. The strict gate cuts subset ASR almost $4.5\times$ (48.6% to 11.4%). The cost is Benign Utility dropping from 100% to 80%: two benign tasks break because they legitimately copy a meeting participant’s email address from an earlier message into `create_calendar_event.participants` — the same data pattern the attack uses, but with no malicious intent.

T2 → T3 (the research question). Field-sensitive gating lowers subset ASR further (11.4% to 8.6%) and raises Utility Under Attack (34.3% to 36.4%), because it stops blocking calls whose only tainted argument is a *content* field (e.g. an email body quoting injected text) — 496 blocked calls vs. 579 under T2. **However, Benign Utility is not recovered:** both policies break the same two benign tasks, because the failures live in the **participants** field, which is sensitive under both policies (it is the same field an attacker uses to redirect invites). So the trade-off between T2 and T3 is close to degenerate on this suite: T3 is marginally safer and marginally more useful under attack, but no less intrusive to the benign user. The bottleneck is precision on *routing* fields, not content fields — a limitation a purely provenance-based gate cannot resolve.

Residual attacks. Every attack that survives T2 (8/140) and T3 (6/140) is the same goal: deleting a file by a short numeric id. The cause is in T1, not the gate — the id is shorter than the 4-character minimum-match threshold, so the taint layer never labels it untrusted and both gates see a trusted argument. This is a documented, conscious trade-off (the threshold prevents false matches on short strings), not a hidden gap.

3 Progress Compared to the Plan

Stage	Description	Status
T0	Baseline without defense	Completed
T1	Tag-only taint tracking	Completed (full eval, traces, leakage metric)
T2	Strict gate	Completed (full eval, false-positive analysis)
T3	Field-sensitive gate	Completed (full eval, answers research question)
T4	Rule-based sanitizer	Optional — not attempted

The previous report claimed T1 was ~60–70% done, and the code was not verifiable in the repository. Since then: the repository is complete and pushed; T1, T2, and T3 are each 100% done including full 150-case evaluations; T0 reporting is separated into the three evaluations; the metric-semantics bug is fixed and regression-tested; and the missing **src/agent/** directory was added. All ten artifacts on the reviewer’s checklist are delivered: clean repository layout, real T1 integration, real traces, T1 non-regression vs. T0, taint leakage metric, T2 strict gate with a privileged-tool list, T3 field-sensitive gate with a sensitive-fields table, the T0–T3 comparison with primary/secondary ASR and overblock rate, unit tests (propagation / strict / field / metric non-regression), and reproducibility (pinned dependencies, exact commands, model, benchmark version, task list, case-count asserts).

4 Plan for the Final Report

The technical ladder T0–T3 is complete; the remaining work is the final write-up and, if time allows, one optional extension:

1. **Final report text:** idea → threat model → per-stage results → research-question answer → limitations → future work. Most of this content already exists in the results summaries and the project site and needs consolidation.
2. **Limitations / future work** (documented, not hidden): (a) the short-id bypass of the minimum-match threshold, potentially closable with provenance-aware exact-field matching instead of substring matching; (b) the **participants**/routing false positives, which need a policy beyond provenance (e.g. an address-book allowlist).

3. **Optional T4:** a lightweight rule-based sanitizer or allowlist for routing fields, attempted only if time remains — it is out of the required scope.

5 Conclusion

The project now has a reproducible T0–T3 evaluation on a single model, a full metric comparison, and documented false positives and residual attacks. The strict gate (T2) sharply reduces attack success at a measurable utility cost; the field-sensitive gate (T3) is slightly safer but does not recover that cost, because the remaining benign failures are concentrated in routing fields that both policies must treat as sensitive. This honest answer — rather than “T3 is strictly better” — is the main substantive result, and it sets up the final report’s discussion of what a provenance-only gate can and cannot achieve.